



Metropolitan University, Sylhet

Department of Computer Science & Engineering

Lab Report – 05:

0/1 Knapsack Problem using Dynamic Programming

Course Title: Algorithm Design and Analysis Lab

Course Code: CSE 232

Submitted To:

Honourable Sir, **Rishad Amin Pulok**
Senior Lecturer
Department of CSE
Metropolitan University, Sylhet

Submitted By:

Mahdi Hasan Shuvo
Student ID: 251-115-030
Batch: CSE 62nd (A)

Date of Submission:

28th February 2026

problem:

0/1 Knapsack Problem using Dynamic Programming

CODE :

```
// 0/1 Knapsack Problem using Dynamic Programming
// Name: Mahdi Hasan Shuvo | ID: 251-115-030 | Batch: CSE 62nd (A)

#include<iostream>
#include<vector>
#include<algorithm>
using namespace std;
struct Item
{
    int weight;
    int profit;
};
int Knapsakprofit(int n, int capacity, vector <Item> &items, vector <int>
&selected)
{
    vector <vector <int>> Table(n+1, vector <int> (capacity+1, 0));
    for (int i = 1; i <= n; i++)
    {
        for (int c = 1; c <= capacity; c++)
        {
            if (items[i-1].weight <= c)
            {
                Table[i][c] = max(Table[i-1][c], items[i-1].profit + Table[i-
1][c-items[i-1].weight]);
            }
            else
            {
                Table[i][c] = Table[i-1][c];
            }
        }
    }

    int i = n;
    int j = capacity;

    while(j > 0 && i > 0)
```

```

        {
            if ((Table[i][j] != Table[i-1][j]) && items[i-1].weight <= j)
            {
                selected[i-1] = 1;
                j -= items[i-1].weight;
                i--;
            }
            else
            {
                i--;
            }
        }

        return Table[n][capacity];
    }

int main()
{
    int n;
    cout << "Enter numbers items: ";
    cin >> n;

    vector <Item> items(n);

    for (int i = 0; i < n; i++)
    {
        cout << "Weight -" << i+1 << ": ";
        cin >> items[i].weight;
        cout << "Profit -" << i+1 << ": ";
        cin >> items[i].profit;
    }

    int capacity;
    cout << "Enter capacity: ";
    cin >> capacity;

    int mx_profit = 0;

    vector <int> selected(n, 0);

    mx_profit = Knapsakprofit(n, capacity, items, selected);

```

```

    cout << "Maximum possible profit: " << mx_profit << endl;

    cout << "Selected items respectively: ";
    for (int i = 0; i < n; i++)
    {
        cout << selected[i] << " ";
    }

    return 0;
}

```

Output Output Output

OUTPUT:

```

66     for (int i = 0; i < n; i++)
67     {
        PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
        Weight -1: 2
        Enter numbers items: 5
        Enter numbers items: 5
        Profit -1: 3
        Weight -2: 4
        Profit -2: 5
        Weight -3: 6
        Profit -3: 7
        Weight -4: 2
        Profit -4: 4
        Weight -5: 6
        Profit -5: 7
        Enter capacity: 11
        Maximum possible profit: 14
        Selected items respectively: 1 0 1 1 0
        PS G:\CSE-62\AD&A>

```

Thank You sir